

Reviewer Pack — Minimal Rank of tropicalized algebraic divisors over finite ...

Entry 88ed206dafbd · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-25 02:39:21 UTC

Minimal Rank of tropicalized algebraic divisors over finite fields vs degree-d SOS approximation for max-CUT

Entry ID: 88ed206dafbd **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-25 02:39:21 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Tropical Geometry) **Field B** (complexity object): Complexity Theory: SOS Approximation for Max-CUT

Statement:

[‘For a tropical polynomial f defined over a finite field F_q with q elements, let $D(f)$ be the algebraic divisor of its zero set in the projective space. Then, any degree- d SOS approximation algorithm for max-CUT with an approximation ratio better than 0.878 must have a moment matrix M associated with f such that its rank is at least R , where $R = q + 1$ if f has no roots and $R > q + 1$ otherwise.’, ‘Equivalently, for any degree- d SOS polynomial G approximating the max-CUT problem, there exists a tropical polynomial f such that the algebraic divisor of its zero set has rank at least R , where R is defined as above.’]

Rationale (proposer’s reasoning):

[‘The minimal rank of tropicalized algebraic divisors could capture essential structural properties of the zero sets of tropical polynomials associated with max-CUT instances. If this property correlates with the approximation quality of SOS algorithms, it may expose a deep connection between tropical geometry and computational complexity.’, ‘Since SOS approximation for max-CUT is related to semidefinite programming, which involves solving systems of polynomial equations over the real numbers, exploring its relation to algebraic divisors in a finite field setting could lead to new insights into the complexity of these problems.’]

Taxonomy category: SOS_DEGREE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 7c871d3110560894

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all generated max-CUT instances with $n \leq 40$ variables and using a degree- d SOS approximation algorithm, the rank of the algebraic divisor $D(f)$

of the tropical polynomial f is at least R where $R = q + 1$ if f has no roots and $R > q + 1$ otherwise. The conjecture is falsified if there exists any instance with the aforementioned conditions where the rank of $D(f)$ is less than R .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'tropical geometry' AND 'SOS approximation for Max-CUT' AND 'algebraic divisor' - 'finite field' AND 'degree-d SOS approximation' AND 'max-CUT problem' AND 'algebraic geometry' - 'moment matrix' AND 'rank of tropical polynomial' AND 'approximation ratio better than 0.878'

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(A):
    m, n = len(A), len(A[0])
    for i in range(m):
        max_row = i
        for j in range(i + 1, m):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]
        if A[i][i] == 0:
            continue
        for j in range(n):
            A[i][j] /= A[i][i]
        for j in range(m):
            if j != i and A[j][i] != 0:
                factor = A[j][i]
                for k in range(n):
                    A[j][k] -= factor * A[i][k]

def matrix_multiply(A, B):
    m, n, p = len(A), len(B), len(B[0])
```

```

C = [[0] * p for _ in range(m)]
for i in range(m):
    for j in range(p):
        for k in range(n):
            C[i][j] += A[i][k] * B[k][j]
return C

def rank_of_matrix(A):
    A_copy = [row[:] for row in A]
    gaussian_elimination(A_copy)
    rank = 0
    for row in A_copy:
        if any(row):
            rank += 1
    return rank

def construct_tropical_polynomial(edges, q):
    n = len(edges) + 1
    f = [0] * (q ** n)
    for u, v in edges:
        f[u * q + v] = -math.inf
    return f

def max_cut_approximation(n, q):
    # Placeholder for actual max-CUT approximation algorithm
    # This is a dummy implementation that always returns a valid solution
    return [random.choice([0, 1]) for _ in range(n)]

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    q = random.randint(2, 5)
    edges = [(i, j) for i in range(n) for j in range(i + 1, n)]
    f = construct_tropical_polynomial(edges, q)
    rank = rank_of_matrix([f])
    G = max_cut_approximation(n, q)
    ratio = sum(G) / n
    metric_value = rank * ratio
    instances_tested = 1
    conjecture_holds = rank >= q + 1
    counterexample = "" if conjecture_holds else "rank_too_low"
    return {
        "metric_name": "Rank of Tropical Polynomial",
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else list(range(2, 30))
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")

```

```

results.append(result)

mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample='rank_too_low' first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE insufficient_support")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a0feed8f.py", line 92, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a0feed8f.py", line 71, in run_trial
    f = construct_tropical_polynomial(edges, q)
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a0feed8f.py", line 56, in construct_tropical_polynomial
    f = [0] * (q ** n)
          ^^^^^^^^^
OverflowError: cannot fit 'int' into an index-sized integer

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing any data, which means we cannot verify the conjecture's conditions. | next: Re-run the test with appropriate error handling to ensure it completes and produces results.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11633
2	propose	ollama_remote	glm4:latest	0	0	11764
3	preregistration	ollama_remote	glm4:latest	0	0	5960
4	novelty	ollama_remote	glm4:latest	0	0	5031
5	novelty	ollama_remote	glm4:latest	0	0	6133
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13341
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9023
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10777
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10941
10	judge	ollama_remote	glm4:latest	0	0	11599

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 96200 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/88ed206dafbd.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/88ed206dafbd.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/88ed206dafbd.tar.gz` (if generated)